



OS Kernel, Instruction Execution and Interrupt Processing

Rab Nawaz Khan Jadoon

Department of Computer Science

DCS

COMSATS Institute of
Information Technology

Lecturer

COMSATS Lahore

Pakistan

Operating System Concepts

Operating System Kernel

- A kernel is a central component of an operating system.
 - It acts as an interface between the user applications and the hardware.
 - The sole aim of the kernel is to manage the communication between the software (user level applications) and the hardware (CPU, disk memory etc).
 - The main tasks of the kernel are :
 - Process management
 - Device management
 - Memory management
 - Interrupt handling, I/O communication and File system.



■ Types of kernels

- May be classified mainly in two categories
 - Monolithic (Mono → single, Lithic → layer)
 - Micro Kernel.

■ Monolithic

- Older approach
 - Basic system services like process and memory management, interrupt handling etc were packed into a single module in kernel space.
- Some serious problems like,
 - Size was huge, Poor maintainability (bug fixing and addition of new features resulted in recompilation of the whole kernel code which could consume hours.
 - E.g. MS-DOS

- **Modern Kernel approach (Modular Approach)**
 - Kernel consists of different modules which can be dynamically loaded and un-loaded.
 - Easy extension of OS's capabilities.
 - Maintainability became so easy as only the concerned module needs to be loaded and unloaded every time if there is change or bug fix in a particular module.
 - **LINUX** follows the Monolithic Modular Approach.

- This solve the problem of ever growing size of kernel code which we could not control in monolithic approach.
 - This architecture allows some basic services like,
 - Device driver management
 - Protocol stack
 - File system etc. to run in user space.
 - It reduces the kernel size and increases the security and stability of OS.
 - E.g. if network service crashes due to the buffer overflow, then only the networking services memory would be corrupted, leaving the rest of the system still functional.

- The important features include,
 - Managing memory protection
 - Process scheduling
 - Inter process communication (IPC)
 - Set of methods for the exchange of data among multiple threads in one or more processes.
- QNX follows the microkernel approach.
 - **QNX** is a commercial Unix-like real-time operating system (RTOS), aimed primarily at the embedded systems market, i.e. BlackBerry

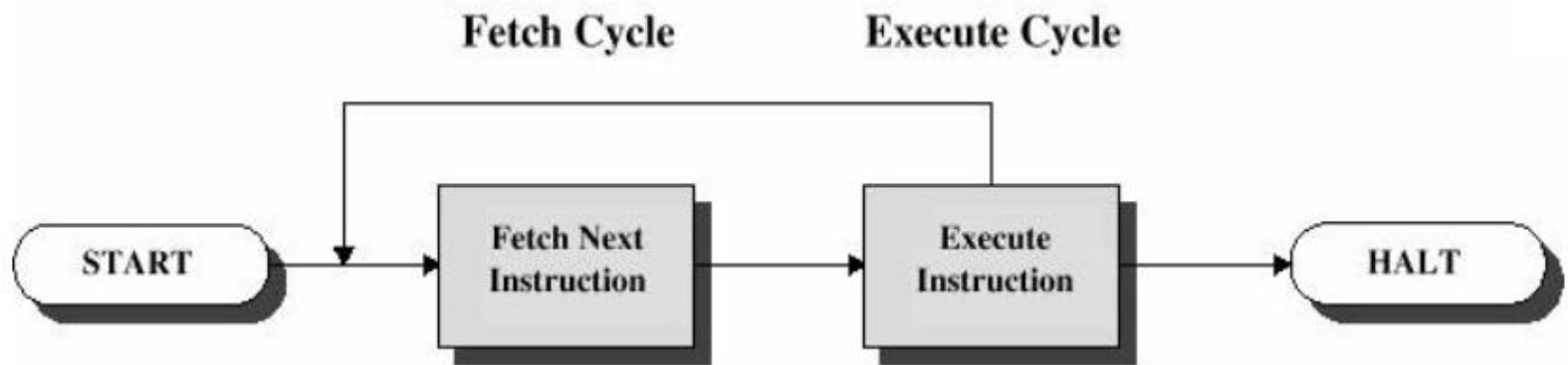
Instruction Execution

- Once a program is in memory it has to be executed.
 - To do this, each instruction must be looked at, decoded and acted upon in turn until the program is completed.
 - This is achieved by the use of what is termed the “instruction execution cycle”, which is the cycle by which each instruction in turn is processed.

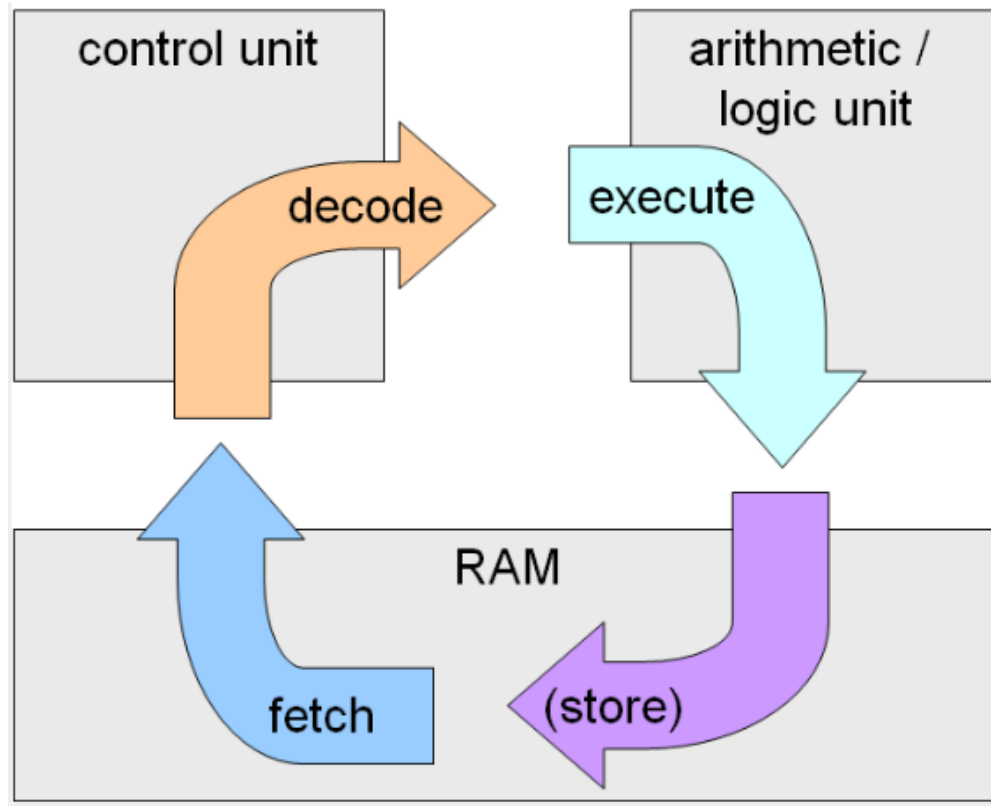
Instruction Execution(IE)

- The process in which CPU reads instruction from memory and execute each instruction, called IE.
 - The processing required for single instruction called instruction cycle.
 - At the beginning of each instruction cycle, CPU fetches instruction from memory.
 - PC holds the address of the next instruction to be executed.
 - The fetched instruction is loaded into IR.

Instruction execution cycle



Instruction execution cycle



- There are two types of jobs

- I/O limited jobs

- If in an application most of the tasks are I/O oriented then it is called I/O jobs.
 - For (k=0;k<=5;k++)
cout<<k;

- CPU limited jobs

- If a program only needs computation/processing then this is called CPU limited jobs.
 - Sum=0;
 - For (k=0;k<=5;k++)
{sum=sum+k;
cout<<sum;}

Interrupt processing

■ Interrupt

- An Interrupt is an event that alters the sequence in which the processor executes instructions.
- When an I/O device completes an I/O operation some events occurs,
 - The device issue an interrupt signal to the CPU.
 - The processor finishes the execution of the current instruction before responding to the interrupt and save the state of the interrupted process.
 - The OS analyses the interrupt and passes the control to the appropriate interrupt handling routine.
 - The IHR handles the interrupt.
 - The state of the interrupted process restored.
 - The interrupted process executes.

Interrupt classes

- There are six interrupt classes,
 - Supervisor Call Interrupt (SVC)
 - It is user generated request for a particular system service such as performing IO, obtaining more storage, or communication with system operator.
 - It secure the OS from user interruption. When a user request for a service it must be handled through a SVC.
 - I/O interrupt
 - These are initiated by IO hardware.
 - They signal to the CPU that device has changed.
 - They are caused when an
 - IO completes, IO error occur or when a device ready etc/.

Interrupt classes

■ External interrupts

- These are caused by various events like expiration of quantum.
- Or a signal from processor to another on a multiprocessor environment.

■ Program check interrupt

- When machine language instructions are executed,
- Division by zero, overflow, underflow etc.

■ Restart

- When the user press the restart button of the computer.
- Or when a restart instruction arrives from another processor on multiprocessor system.

Interrupt classes

- **Machine check interrupt**
 - These are caused by the malfunctioning of the hardware.

Context switching

- A **context switch** is the computing process of storing and restoring the state (context) of a CPU so that execution can be resumed from the same point at a later time.
 - This enables multiple processes to share a single CPU.
 - The context switch is an essential feature of a multitasking operating system.
 - Context switches are usually computationally intensive and much of the design of operating systems is to optimize the use of context switches

Context switching

- When an interrupt occurs, the operating system saves the interrupted process and shift the control to the appropriate first level interrupt handler (FLIH)
 - This is handled through context switching
- Program Status Word (PSW)
 - Control the order of instruction execution and contains the various sates of a process.
 - There are three types of PSW
 - Current PSW, New PSW and Old PSW.

Context switching

■ Current PSW

- The address of the next instruction to be executed are stored/kept in **current PSW**, which indicated the interrupt type currently enabled or disabled.
- The CPU allows enabled interrupts to occur.
- On a uniprocessor system there is only one current PSW, but there are six new PSWs for each interrupt class and six old PSWs, one for each Interrupt class.

Context switching

■ New PSW

- It contains the memory address at which the interrupt handler for that interrupt type resides.
- When an interrupt occur, then the H/W automatically switches PSWs by,
 - Storing the **current PSW** in the **old PSW** for that type of interrupt.
 - Storing the new PSW for that type of interrupt into the **current PSW**.
- After this swap the current PSW now contains the address of the appropriate IH.
- The IH executes and process the interrupt.

Context switching

- When the processing of the interrupt completes, the CPU is dispatched to either the interrupted process or to the highest priority ready process.



Thanks